SALESIA ENTERPRISE · FASE 16

Plan de mantenimiento

Operación, cambios, copias de seguridad y evolución

DOCUMENTO	plan-mantenimiento.md
VERSIÓN	1.0
FECHA	14 de octubre de 2026
PLATAFORMA	React 19 + TypeScript · FastAPI · PostgreSQL 17
DESPLIEGUE	Render (API) · Vercel (web) · Supabase (BD)
ESTADO	Sistema desplegado y verificado

Índice

1. Composición y responsabilidades 3
2. Rutina de mantenimiento 4
    2.1 Diario / semanal 4
    2.2 Mensual 4
    2.3 Trimestral 4
3. Copias de seguridad y recuperación 4
4. Gestión de cambios 5
    4.1 Flujo de trabajo 5
    4.2 Migraciones de esquema 5
    4.3 Versionado 5
5. Dependencias y seguridad 5
6. Verificación tras cada despliegue 6
7. Monitorización 6
8. Plan de evolución (Semana 08 según el plan) 7
9. Soporte al usuario 7

- 1. Composición y responsabilidades
- 2. Rutina de mantenimiento
  - 2.1 Diario / semanal
  - 2.2 Mensual
  - 2.3 Trimestral
- 3. Copias de seguridad y recuperación
- 4. Gestión de cambios
  - 4.1 Flujo de trabajo
  - 4.2 Migraciones de esquema
  - 4.3 Versionado
- 5. Dependencias y seguridad
- 6. Verificación tras cada despliegue
- 7. Monitorización
- 8. Plan de evolución (Semana 08 según el plan)
- 9. Soporte al usuario

Plan de mantenimiento — SalesIA Enterprise

> Procedimientos de operación, evolución y soporte del sistema. Versión 1.0 · Fase 16.

1. Composición y responsabilidades

Área	Componente	Ubicación
Interfaz	React 19 + TypeScript + Vite	`frontend/`
API	FastAPI + SQLAlchemy	`backend/app/`
Lógica de negocio	15 servicios	`backend/app/services/`
Datos	PostgreSQL 17 (Supabase)	`database/migrations/`
Despliegue	Vercel + Render	`vercel.json`, `render.yaml`
Documentación	Markdown + PDF	`docs/`

Regla del proyecto: **la fuente de verdad del esquema es el SQL plano** en `database/migrations/`. El ORM lo refleja, no lo define. No se usa Alembic.

2. Rutina de mantenimiento

2.1 Diario / semanal

- Revisar los **logs de Render** (pestana *Logs*) en busca de errores 500.
- Vigilar los **alertas de Supabase**: uso de CPU, conexiones y espacio.
- Comprobar la **salud de la API**: `bash curl -s https://salesia-enterprise-brqr.onrender.com/health/database`

2.2 Mensual

- **Rotar secretos**: generar clave nueva con `openssl rand -hex 32` y actualizar `JWT_SECRET_KEY` en Render (invalida las sesiones abiertas).
- **Revisar la auditoría** (`/api/audit`) para detectar accesos anómalos.
- **Analizar el crecimiento** de las tablas `sales` , `audit_logs` , `observations` y `inventory_movements` , que son las de mayor volumen.
- **Actualizar dependencias** (ver §5).

2.3 Trimestral

- Revisar **usuarios activos y roles**: eliminar o desactivar cuentas huérfanas.
- **Copias de seguridad**: verificar el *point-in-time recovery* de Supabase y probar una restauración real en el entorno de desarrollo.
- Revisar el **plan de despliegue** (límites de Render/Vercel y coste).

3. Copias de seguridad y recuperación

Qué	Cómo	Frecuencia
BD completa	PITR de Supabase (automático)	Continua
Snapshot manual	`pg_dump` antes de cada migración	Por evento
Esquema + seeds	Versionados en git (`database/`)	Continua

Exportar una copia manual

```
pg_dump "$DATABASE_URL" --no-owner --no-privileges -Fc -f salesia_$(date +%F).dump
```

Restaurar en un entorno de pruebas

```
pg_restore --clean --if-exists -d "$DATABASE_URL_PRUEBAS" salesia_2026-10-14.dump
```

⚠️ **Nunca restaurar un volcado de producción sobre producción** sin confirmación expresa del responsable. Para una BD de desarrollo, la alternativa rápida es reseedear desde cero: `cd backend && .venv/bin/python seed_demo.py`.

4. Gestión de cambios

4.1 Flujo de trabajo

1. Crear rama desde `main`: `git checkout -b feat/nombre-del-cambio`.
2. Implementar con sus pruebas.
3. **Quality gates obligatorios antes de cualquier commit**: `bash cd backend && .venv/bin/python -m pytest tests -q # 115 pruebas cd frontend && npm run lint && npm test && npm run build`
4. Commit con asunto en español y cuerpo en viñetas.
5. Revisión y fusión a `main`. Vercel y Render redespliegan automáticamente.

4.2 Migraciones de esquema

1. **Nunca editar** una migración ya aplicada: crear un nuevo fichero `database/migrations/00N_descripcion.sql`.
2. Probar en una BD de desarrollo (`seed_demo.py` deja una BD limpia).
3. **Volcado de seguridad** antes de aplicar en producción.
4. Aplicar con transacción: `bash psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f database/migrations/00N_....sql`
5. Verificar con la cadena de comprobación (§6).

⚠️ Si la migración inserta filas con `id` explícito, hay que **realignar las secuencias** (`database/seeds/998_reset_sequences.sql`). Sin esto, el siguiente INSERT del API devuelve `409 duplicate key`.

4.3 Versionado

El proyecto sigue **SemVer** (`MAJOR.MINOR.PATCH`):

- **MAJOR**: cambios incompatibles en la API o el esquema.
- **MINOR**: funcionalidad nueva y compatible.
- **PATCH**: correcciones y mejoras internas.

5. Dependencias y seguridad

Acción	Cuándo	Cómo
Actualizar dependencias	Mensual	`npm update` / `pip install -U -r requirements.txt`
Escanear vulnerabilidades npm	Semanal	`npm audit`

Acción	Cuándo	Cómo
Escanear vulnerabilidades pip	Mensual	`pip-audit -r backend/requirements.txt`
Actualizar Python	Semestral	Soporte de versiones: 3.12 LTS
Actualizar Node	Semestral	LTS activa (22)

Protocolo de vulnerabilidad

1. Confirmar la afectación y su criticidad.
2. Corregir o aplicar el parche en una rama.
3. Ejecutar la cadena de verificación completa.
4. Desplegar y verificar en producción.
5. Registrar el incidente.

Buenas prácticas vigentes: Argon2 para contraseñas, JWT con expiración de 60 min, secretos **sólo** en variables de entorno (nunca en git), CORS restringido y validación de entrada con Pydantic.

6. Verificación tras cada despliegue

Comprobación mínima (2 minutos)

```
curl -s https://salesia-enterprise-brqr.onrender.com/health          # healthy
curl -s https://salesia-enterprise-brqr.onrender.com/health/database # PostgreSQL 17
```

Y en el navegador: login correcto y carga del Dashboard.

Verificación completa

Sobre una BD de desarrollo reseedada, ejecutar en este orden:

```
cd backend
for n in 12 11 10 09 08 13; do PYTHONPATH=$PWD .venv/bin/python verify_fase$n.py; done
```

> **El orden importa** y la BD debe estar fresca: los scripts dejan datos residuales y la Fase 11 calcula sobre deltas. Ver evidencias-pruebas.md.

7. Monitorización

Señal	Herramienta	Umbral de alerta
Disponibilidad de la API	`/health` (Render)	Cualquier 5xx sostenido
Latencia de la base	`/health/database`	> 2 s de forma recurrente

Señal	Herramienta	Umbral de alerta
Errores 500	Logs de Render	> 3 en 10 minutos
Conexiones a BD	Supabase dashboard	> 80 % del límite
Espacio en disco	Supabase dashboard	> 80 %
Uso de función	Vercel dashboard	> 80 % del cuota

Incidentes conocidos y su solución están recogidos en la tabla de troubleshooting del manual técnico.

8. Plan de evolución (Semana 08 según el plan)

La arquitectura admite ampliar el motor estadístico sin tocar el esquema existente: basta añadir métricas en `stats_service.py` y, si procede, una regla en `insight_service.py`.

Funcionalidad	Punto de entrada
Varianza y desviación estándar	`stats_service.py` · `_distribution_stats`
Distribuciones y probabilidades	`stats_service.py` · `run_probability`
Pruebas de hipóstasis y p-valor	Nuevo endpoint en `api/routes/analyses.py`
Esperanza matemática	`stats_service.py` · `run_random_variable`
Modelos predictivos	Nuevo servicio + tabla de resultados

Cada ampliación debe: registrar su análisis en `statistical_analyses`, exponer el resultado en `statistical_results`, respetar el aislamiento por `company_id` y añadir sus pruebas a las suites existentes.

9. Soporte al usuario

Nivel	Responsable	Respuesta
Nivel 1 — dudas de uso	Administrador del sistema	Consulta del manual de usuario
Nivel 2 — incidencias funcionales	Equipo técnico	Registro con usuario, módulo, hora y mensaje
Nivel 3 — fallos técnicos	Responsable técnico	Logs de Render/Vercel + `audit_logs`

Para cualquier incidencia se necesita: **usuario y rol**, **módulo**, **hora** aproximada y **mensaje** mostrado. Con eso se localiza el suceso en la auditoría.